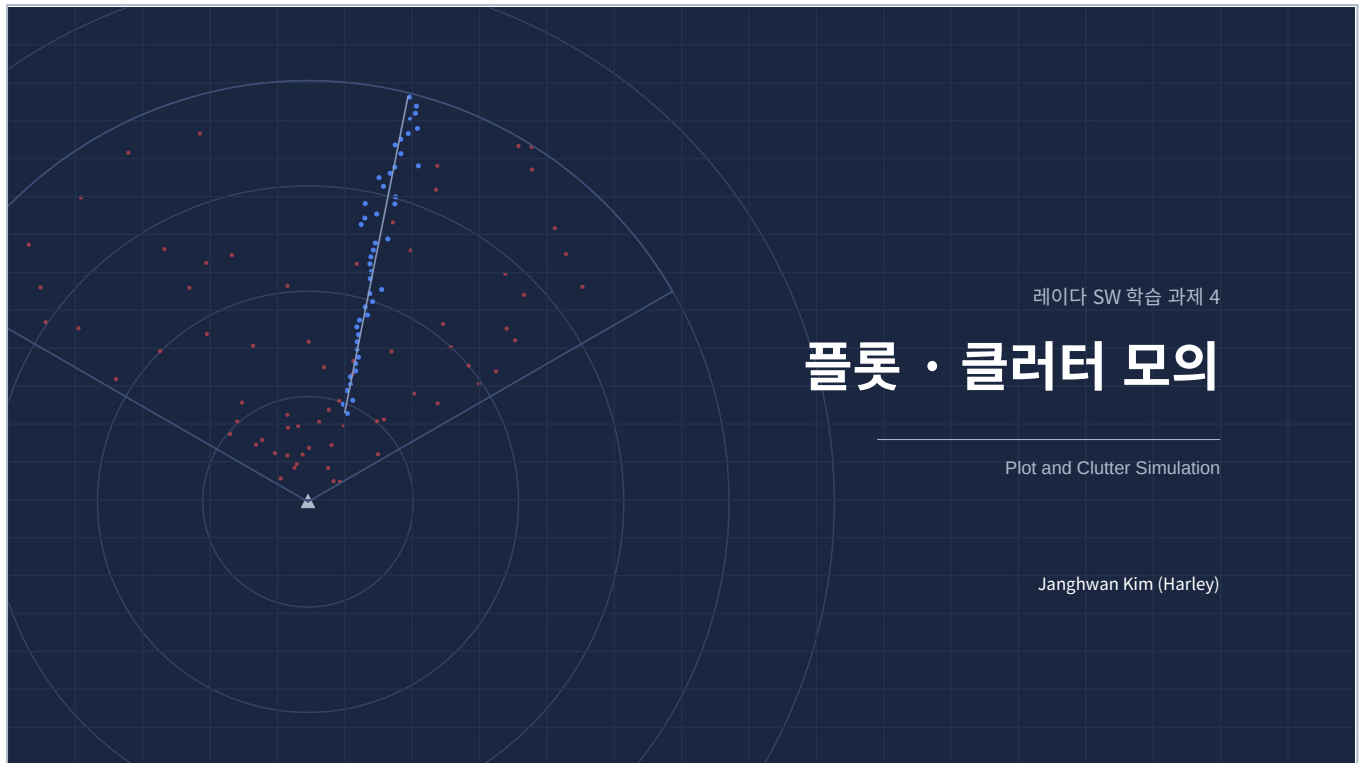




표지

발표 대본

안녕하십니까. 플롯과 클러터 모의를 발표하겠습니다.

레이다가 표적을 쏜 것 같은 측정값을 컴퓨터로 만드는 과제입니다.

INDEX		
01	플롯	3
02	플롯 모의	5
03	클러터	8
04	구현과 핵심 코드	10
05	실습 결과	16

2

목차

발표 대본

순서는 다섯 부분입니다.

플롯, 플롯 모의, 클러터의 개념을 먼저 설명드리고, 네 번째로 구현과 핵심 코드를, 다섯 번째로 실습 결과를 보여 드리겠습니다.

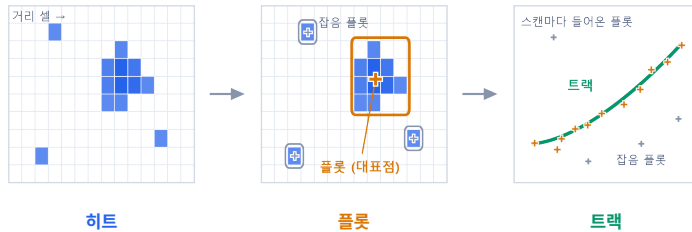
01. 플롯

1.1 플롯은 레이다가 표적을 한 번 쏜 결과입니다

레이다 처리 흐름에서의 위치



히트 → 플롯 → 트랙

**히트** 신호가 문턱을 넘은 칸 하나. 표적 하나가 여러 히트를 낸다**플롯** 히트를 묶은 대표점, 거리 · 방위각 · 고각을 가진다**트랙** 플롯을 시간 순서로 이은 궤적 (다음 단계)**드웰** 빔이 한 방향에 머무는 시간. 드웰마다 플롯이 나온다

3

01. 플롯 | 1.1 플롯은 레이다가 표적을 한 번 쏜 결과입니다

발표 대본

첫 번째, 플롯입니다.

플롯은 레이다가 표적을 한 번 쏜 결과입니다.

위 흐름을 보시면, 과제 3에서는 표적의 실제 궤적을 만들었습니다.

이번 과제는 그 궤적을 레이다가 쏜 것처럼 바꾸는 측정 모의이고, 여기서 만든 플롯을 다음 단계인 추적 필터가 받습니다.

아래 그림처럼 신호가 문턱을 넘은 칸을 히트라고 합니다.

붙어 있는 히트를 묶어 대표점 하나로 만든 것이 플롯이고, 플롯을 시간 순서로 이은 것이 트랙입니다.

빔이 한 방향에 머무는 시간을 드웰이라고 하며, 드웰마다 플롯이 나옵니다.

01. 플롯

1.2 플롯을 만드는 길은 두 가지, 이번 과제는 신호처리를 건너뛰입니다

① 실제 레이더

신호를 다듬고 키운다

문턱을 넘은 칸을 고른다

묶어서 위치를 정한다

1 IQ 데이터

2 펄스압축

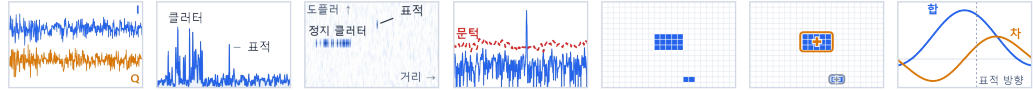
3 도플러 처리

4 CFAR

5 Hit 추출

6 Hit 클러스터링

7 모노펄스

수신 신호 (IQ) → 신호처리 7단계 → **드웰 플롯** (거리 · 방위각 · 고각) 필요한 것: 파형, 안테나, 신호처리 전체

② 플롯 모의

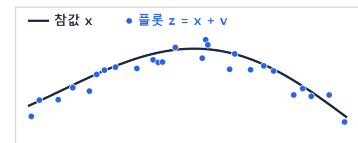
표적 참값 x

신호 생성 · 신호처리 생략

플롯 $z = x + v$ 필요한 것
표준편차 3개

건너뛰어도 되는 이유

- 추적 필터가 받는 것은 플롯뿐이다
- 신호처리를 거친 플롯도 참값 + 작은 오차 → 오차 크기만 맞추면 같은 입력



4

01. 플롯 | 1.2 플롯을 만드는 길은 두 가지, 이번 과제는 신호처리를 건너뛰입니다

발표 대본

플롯을 만드는 길은 두 가지입니다.

위쪽은 실제 레이더로, 수신 신호를 7단계로 처리합니다.

1번에서 3번은 신호를 다듬고 키우고, 4번과 5번은 문턱을 넘은 칸을 고르고, 6번과 7번은 그 칸을 묶고 모노펄스로 각도를 재서 위치를 정합니다.

아래쪽이 이번 과제의 플롯 모의입니다.

신호 생성과 신호처리를 건너뛰고, 참값에 잡음을 바로 더합니다.

이래도 되는 이유는 두 가지입니다.

첫째, 추적 필터가 받는 것은 플롯뿐입니다.

둘째, 신호처리를 거친 플롯도 결국 참값에 작은 오차가 더해진 값입니다.

그래서 오차 크기, 즉 표준편차 세 개만 맞추면 같은 입력을 만들 수 있습니다.

02. 플롯 모의

2.1 플롯 모의는 SNR → 표준편차 → 잡음 → 플롯 순서입니다



단계 1 SNR 은 원래 레이더 방정식으로 구한다

$$SNR = \frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 R^4 k T_0 B F L}$$

빔폭 → 안테나 이득
대역폭 → 잡음 전력

실제 SNR 은 거리의 네제곱에 반비례해서
표적이 다가오면 커진다.
실습은 SNR 을 20 으로 고정한다.

SNR 은 선형값

신호 세기 ÷ 잡음 세기. SNR 20 = 신호가 잡음의 20배

참값 x 는 안테나에서 본 거리 · 방위각 · 고각

과제 3 의 표적 위치에서 구한다 → 4.5 에서 코드와 함께

5

02. 플롯 모의 | 2.1 플롯 모의는 SNR → 표준편차 → 잡음 → 플롯 순서입니다

발표 대본

두 번째, 플롯 모의입니다.

위의 네 단계로 진행합니다.

SNR 을 정하고, 표준편차 p 를 계산하고, 잡음을 뽑아서, 참값에 더합니다.

SNR 은 신호 세기를 잡음 세기로 나눈 선형값입니다.

원래는 왼쪽 레이더 방정식으로 구하고 표적이 다가오면 커지지만, 실습은 20, 즉 신호가 잡음의 20배로 고정합니다.

참값을 구하는 방법은 4.5 에서 코드와 함께 보여 드리겠습니다.

02. 플롯 모의

2.2 측정 오차 p 는 분해능을 $\sqrt{(2 \cdot \text{SNR})}$ 로 나눈 값입니다

식: 세 개 모두 같은 꼴

$$p = \text{분해능} \div \sqrt{(2 \cdot \text{SNR})}$$

각도는 모노펄스 상수 KM 으로 한 번 더 나눈다

거리

$$p_{\text{rng}} = 29.98 \text{ m} \div \sqrt{(2 \cdot \text{SNR})}$$

$$\text{분해능} = c \div 2B$$

방위각

$$p_{\text{azi}} = 5.14^\circ \div (1.7 \times \sqrt{(2 \cdot \text{SNR})})$$

$$\text{분해능} = \text{빔폭}, 1.7 = \text{KM}$$

고각

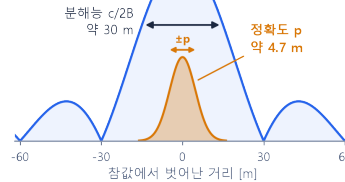
$$p_{\text{ele}} = 5.31^\circ \div (1.7 \times \sqrt{(2 \cdot \text{SNR})})$$

$$\text{분해능} = \text{빔폭}, 1.7 = \text{KM}$$

c 광속, B 대역폭 5 MHz, KM 모노펄스 기술기 상수
SNR 은 선형값

분해능과 정확도

압축된 펄스



분해능

두 표적을 따로 볼 수 있는 간격 (거리 약 30 m)

정확도 = p

한 표적의 측정값이 흔들리는 폭 (약 4.7 m)

과제 값 넣기

$$\sqrt{(2 \times 20)} = \sqrt{40} = 6.32$$

거리

$$29.98 \text{ m} \div 6.32$$

4.74 m

방위각

$$5.14^\circ \div (1.7 \times 6.32)$$

0.478°

고각

$$5.31^\circ \div (1.7 \times 6.32)$$

0.494°

대역폭이 넓고, 빔이 좁고, SNR 이 클수록
 p 가 작다 = 더 정확하다

6

02. 플롯 모의 | 2.2 측정 오차 p 는 분해능을 $\sqrt{(2 \cdot \text{SNR})}$ 로 나눈 값입니다

발표 대본

핵심인 표준편차 p 입니다. p 는 측정값이 참값에서 보통 얼마나 벗어나는지를 나타냅니다.

왼쪽 식은 세 개지만 모양이 같습니다.

분해능을 루트 2 SNR 로 나누고, 각도는 모노펄스 상수 KM 으로 한 번 더 나눕니다.

가운데 그림처럼 분해능은 두 표적을 따로 볼 수 있는 간격으로, 거리는 약 30 m 입니다.

한 표적의 측정값은 이보다 훨씬 좁게 흔들리는데, 그 폭이 p 입니다.

오른쪽에 과제 값을 넣었습니다.

루트 40 은 6.32 이고, 거리는 29.98 m 를 6.32 로 나눠 4.74 m 입니다.

방위각은 5.14도를 1.7 곱하기 6.32 로 나눠 0.478도, 고각은 0.494도입니다.

대역폭이 넓고, 빔이 좁고, SNR 이 클수록 p 가 작아집니다.

02. 플롯 모의

2.3 거리 · 방위각 · 고각마다 잡음을 따로 뽑아 더합니다

$$Z = X + V$$

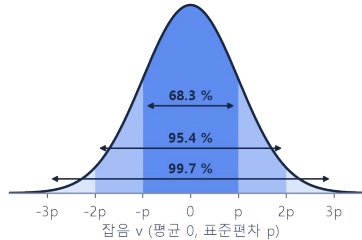
$$v \sim N(0, p^2)$$

평균 0, 표준편차 p 인 정규분포

x 참값

v 잡음

z 플롯 (레이다가 켜 값)



$\pm p$ 안에 68 %, $\pm 3p$ 안에 99.7 %

성분마다 따로 뽑는다

거리 GAUSS(0, 4.74 m)

방위각 GAUSS(0, 0.478°)

고각 GAUSS(0, 0.494°)

세 성분의 잡음은 서로 독립

그래서 플롯 z 는 참값 x 주위에 흩어지고, 흩어지는 폭이 p 다

실제 숫자로 만든 첫 플롯과 3,000개 확인 결과는 실습 2 (5.2) 에서

7

02. 플롯 모의 | 2.3 거리 · 방위각 · 고각마다 잡음을 따로 뽑아 더합니다

발표 대본

이 p 로 잡음을 뽑습니다.

왼쪽 식처럼 플롯 z 는 참값 x 에 잡음 v 를 더한 값이고, v 는 평균 0, 표준편차 p 인 정규분포를 따릅니다.

가운데 그림처럼 뽑은 값의 68 퍼센트는 플러스 마이너스 p 안에 들어옵니다.

오른쪽처럼 거리, 방위각, 고각마다 GAUSS 함수로 따로 뽑고, 세 잡음은 서로 독립입니다.

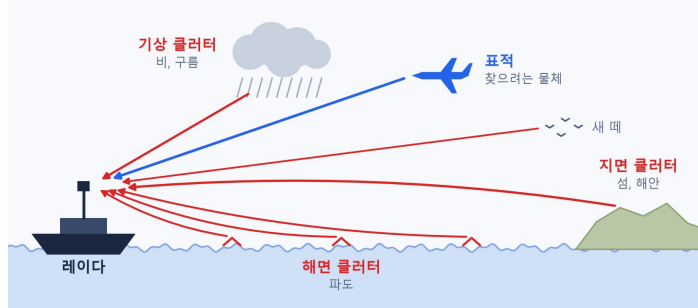
그래서 플롯은 참값 주위에 폭 p 로 흩어집니다.

실제 숫자로 만든 첫 플롯은 실습 결과에서 보여 드리겠습니다.

03. 클러스터

3.1 클러스터는 표적이 아닌 것에서 돌아온 탐지입니다

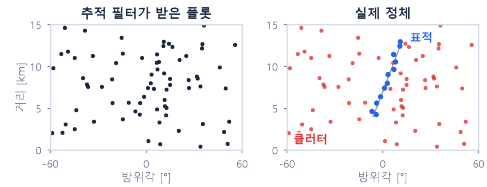
클러스터가 생기는 곳



실제 클러스터를 그대로 만들기는 어렵다

바다 상태 · 지형 · 날씨까지 넣어야 하는 별도 연구 분야

추적 필터가 받는 모습 (예시)



클러스터도 거리 · 방위각 · 고각을 가진 플롯으로 들어온다.
플롯만 봐서는 표적을 구분할 수 없다.

그래서 단순히 만든다

범위 안에 무작위로 생기는 클러스터 → 클러스터 속 추적 시험에 충분

8

03. 클러스터 | 3.1 클러스터는 표적이 아닌 것에서 돌아온 탐지입니다

발표 대본

세 번째, 클러스터입니다.

레이다 전파는 표적에서만 돌아오지 않습니다.

파도, 섬, 비구름, 새 떼에서도 돌아옵니다.

이렇게 표적이 아닌 것에서 생긴 탐지가 클러스터입니다.

클러스터도 거리, 방위각, 고각을 가진 플롯으로 들어오기 때문에, 오른쪽처럼 플롯만 봐서는 어느 것이 표적인지 알 수 없습니다.

실제 클러스터를 그대로 만드는 것은 하나의 연구 분야일 만큼 어려워서, 이번에는 범위 안에 무작위로 생기는 클러스터로 단순히 만듭니다.

03. 클러터

3.2 클러터는 FOV 안에서 균등난수로 뽑습니다

① 개수

스캔마다 5개

② FOV 범위

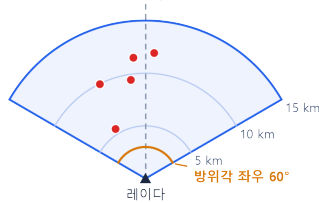
거리 200 m ~ 15 km, 방위각 $\pm 60^\circ$, 고각 $0 \sim 30^\circ$

③ 뽑기

거리 · 방위각 · 고각을 균등난수로 따로

FOV: 레이더가 보는 범위 (과제 값)

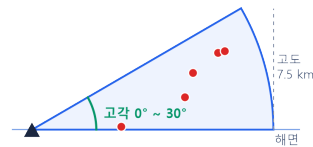
위에서 본 범위



빨간 점 = 한 스캔의 클러터 5개

옆에서 본 범위

● 한 스캔의 클러터 5개



균등난수로 뽑는 법

$$R = R_{min} + (R_{max} - R_{min}) \cdot u, \quad u \sim U(0, 1)$$

균등 난수 u
0 과 1 사이 어느 값이나
같은 확률로 나온다거리 범위에 맞춰 늘린다
 $200 + 14,800 \times 0.37 = 5,676 \text{ m}$ 방위각 = $-60^\circ + 120^\circ \times u$, 고각 = $30^\circ \times u$
스캔마다 새로 뽑고, 표적과는 상관없다

9

03. 클러터 | 3.2 클러터는 FOV 안에서 균등난수로 뽑습니다

발표 대본

클러터 모의는 위의 세 단계입니다.

개수를 정하고, 생길 수 있는 범위인 FOV 를 정하고, 그 안에서 거리, 방위각, 고각을 균등난수로 따로 뽑습니다.

개수는 실습 조건대로 스캔마다 5개입니다.

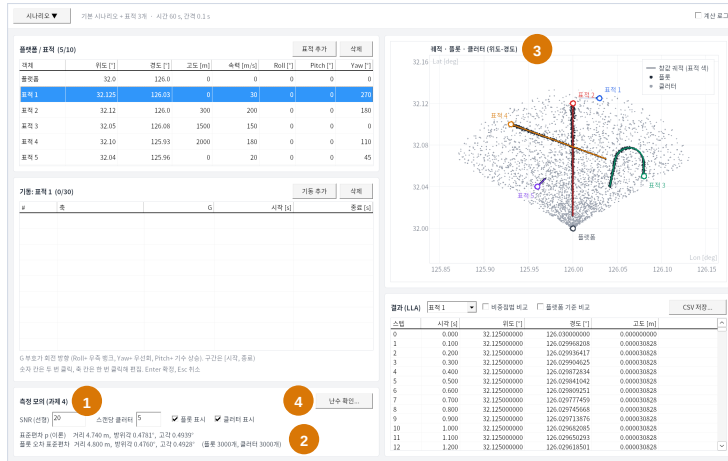
오른쪽 아래처럼 0 과 1 사이 균등난수 u 를 범위에 곱합니다.

예를 들어 u 가 0.37 이면, 거리는 200 m 에 14,800 m 의 0.37 배를 더한 5,676 m 입니다.

클러터는 스캔마다 새로 뽑고, 표적과는 상관이 없습니다.

04. 구현과 핵심 코드

4.1 과제 3 프로그램을 복사해 측정 모의를 더했습니다



TargetSimCore C 라이브러리 (DLL)

과제 3 계산 코드 그대로 + 측정 모의 PlotSim.c

f_Pcs_TestRand 난수 확인 5.1

f_Pcs_CalcSigma 표준편차 p 4.3

f_Pcs_Measure 플롯 · 클러스터 모의 4.4

난수 UNIRAN · GAUSS 는 참고 소스 그대로

TargetSimUI 과제 3 화면에 추가

1 입력: SNR, 스캔당 클러스터 수

2 요약: 이론 p 와 실제 오차

3 지도: 궤적 위에 플롯 · 클러스터

4 난수 확인 창

10

04. 구현과 핵심 코드 | 4.1 과제 3 프로그램을 복사해 측정 모의를 더했습니다

발표 대본

네 번째, 구현과 핵심 코드입니다.

과제 3 에서 표적을 모의했으니, 이번에는 그 표적을 레이다가 재는 단계입니다.

그래서 과제 3 프로그램을 Part 4 폴더에 그대로 복사하고, 그 위에 측정 모의를 더했습니다.

과제 3 원본은 손대지 않았습니다.

계산 라이브러리 TargetSimCore 에는 측정 모의 파일 PlotSim.c 를 더했고, 화면이 부르는 함수는 난수 확인, 표준편차 계산, 측정 모의 세 개입니다.

화면에는 네 가지를 더했습니다.

1번에 SNR 과 스캔당 클러스터 수를 넣으면 바로 다시 모의하고, 2번에 이론 p 와 실제 오차가 나옵니다.

3번 지도는 표적 궤적 위에 플롯과 클러스터를 겹쳐 그리고, 4번 버튼은 난수 확인 창을 엽니다.

실습 조건은 이 화면의 설정 그대로입니다.

맨 위에 시간 60초, 간격 0.1초, 왼쪽 표에 표적 5개, 1번 칸에 SNR 20 과 스캔당 클러스터 5개가 보입니다.

표적은 과제 3 기본 시나리오의 대함, 대공에 3개를 더했고, 번호는 과제 3 과 같습니다.

04. 구현과 핵심 코드

4.2 실행하면 궤적 → 측정 모의 → 정리 → 화면 순서로 계산합니다



상자 = 화면 쪽 (TargetSimUI) 함수, 들여쓰기는 부르는 함수. 파란 글자 = C 라이브러리 (TargetSimCore) 함수

11

04. 구현과 핵심 코드 | 4.2 실행하면 궤적 → 측정 모의 → 정리 → 화면 순서로 계산합니다

발표 대본

먼저 전체 흐름입니다.

왼쪽 상자가 함수이고, 들여쓰는 것이 그 함수가 부르는 함수입니다.

파란 글자는 C 라이브러리 함수입니다.

시나리오를 실행하면 f_RunScenario 가 먼저 1번, 궤적을 과제 3 그대로 계산합니다.

표본은 601개입니다.

이어서 f_RunMeasure 가 측정 모의를 합니다.

CPlotResult 의 f_Run 이 2번, C 라이브러리의 f_Pcs_Measure 를 불러 600 스캔의 참값, 플롯, 클러스터를 만들고, 3번, 오차 표준편차와 지도 좌표를 정리합니다.

4번, 화면 요약 두 줄을 쓰고 f_SetMeasure 로 지도를 다시 그립니다.

SNR 이나 클러스터 수만 바꾸면 f_RunMeasure 부터 다시 하고, 궤적은 그대로 씁니다.

보이기 체크는 지도만 다시 그리고, 난수 확인 버튼은 따로 f_Pcs_TestRand 를 부릅니다.

이제 오른쪽 장 번호 순서로 코드를 보겠습니다.

04. 구현과 핵심 코드

4.3 p 는 2.2 의 식 그대로, 잡음은 참고 소스 GAUSS 로 뱉습니다

NoiseGeneration.c 참고 소스 그대로

```
long long int SEED = 12345;
...
double UNIRAN(void)
{
    long long int M = 2147483647;
    long long int a = 16807;
    long long int xn = 0;
    ...
    xn = SEED;
    xn = abs((a * xn) % M);
    SEED = xn;
    ans = (double)xn / (double)M;
    return ans;
}

double GAUSS(double m, double dev)
{
    int i;
    double a = 0.;
    for (i = 0; i < 12; i++)
    {
        a += UNIRAN();
    }
    return ((a - 6.0) * dev + m);
}
```

UNIRAN (0, 1) 균등난수

$x_n \leftarrow 16807 \times x_n \bmod (2^{31} - 1)$
 돌려주는 값은 $x_n \div M$
 x_n 은 SEED 에 남겨 다음에 이어 씀
 seed 가 같으면 늘 같은 순서

GAUSS 정규난수

UNIRAN 12개 합: 평균 6, 분산 1
 6 을 빼면 평균 0, 분산 1
 (중심극한정리로 종 모양)
 dev 를 곱하고 m 을 더함
 잡음 $v = \text{GAUSS}(0, p)$

PlotSim.c f_Pcs_CalcSigma (2.2 의 식)

```
// 측정 표준편차 p. snr 은 선형값.
...
STRUCT_Coord_Sph f_Pcs_CalcSigma(FLOAT64 snr)
{
    const FLOAT64 root = sqrt(2.0 * snr);
    STRUCT_Coord_Sph st_Sigma;

    st_Sigma.r = LIGHT_SPEED / (2.0 * PCS_BANDWIDTH * root);
    st_Sigma.az = f_Deg_To_Rad(PCS_BEAM_AZI) / (PCS_KM * root);
    st_Sigma.el = f_Deg_To_Rad(PCS_BEAM_ELE) / (PCS_KM * root);

    return st_Sigma;
}
```

SNR 20 을 넣으면 $\text{root} = \sqrt{2 \times 20} = 6.325$

거리	$c \div (2 \times 5 \text{ MHz} \times 6.325)$	4.740 m
방위각	$5.14^\circ \div (1.7 \times 6.325)$	0.4781°
고각	$5.31^\circ \div (1.7 \times 6.325)$	0.4939°

각도는 라디안으로 계산 (f_Deg_To_Rad). 화면 요약 첫 줄과 같은 값

12

04. 구현과 핵심 코드 | 4.3 p 는 2.2 의 식 그대로, 잡음은 참고 소스 GAUSS 로 뱉습니다

발표 대본

먼저 재료인 표준편차와 난수입니다.

오른쪽 f_Pcs_CalcSigma 는 2.2 의 식을 그대로 옮겼습니다.

루트 2 SNR 을 root 로 한 번 구하고, 거리는 광속을 2 곱하기 대역폭 곱하기 root 로 나눕니다.

각도는 빔폭을 라디안으로 바꿔 KM 곱하기 root 로 나눕니다.

SNR 20 을 넣으면 거리 4.740 m, 방위각 0.4781도, 고각 0.4939도이고, 화면 요약 첫 줄과 같은 값입니다.

왼쪽은 참고 소스의 난수 함수 그대로입니다.

UNIRAN 은 이전 값에 16807 을 곱하고, 2의 31승 빼기 1 로 나눈 나머지를 다음 값으로 씁니다.

이 값을 SEED 에 남겨 이어 쓰므로, seed 가 같으면 늘 같은 순서가 나옵니다.

GAUSS 는 UNIRAN 12개를 더해 6 을 빼고, dev 를 곱하고 m 을 더합니다.

12개 합은 평균 6, 분산 1 이라, 6 을 빼면 평균 0, 분산 1 인 종 모양이 됩니다.

그래서 잡음 v 는 GAUSS 0, p 로 뱉습니다.

04. 구현과 핵심 코드

4.4 f_Pcs_Measure 가 스캔마다 참값 · 플롯 · 클러스터를 만듭니다

```

VOID f_Pcs_Measure(ST_PcsScan *st_Scan, const ST_SimSample *st_Sample, INT32 nScanNum, ...)
{
    1 const STRUCT_Coord_Sph st_Sigma = f_Pcs_CalcSigma(snr);
    ...
    2 changeSEED(PCS_SEED);
    for (nScan = 0; nScan < nScanNum; nScan++)
    {
        st_In = &st_Sample[nScan];
        st_Radar = &st_In->st_Platform;
        st_Out = &st_Scan[nScan];
        ...
        3 for (nTarget = 0; nTarget < st_In->nTargetNum; nTarget++)
        {
            // 참값 x
            4 st_Out->st_True[nTarget] = f_Pcs_EcefToSph(&st_In->st_Target[nTarget].st_PosEcef, st_Radar);
            // 플롯 z = x + v, v ~ N(0, p^2)
            st_Out->st_Plot[nTarget].r = st_Out->st_True[nTarget].r + GAUSS(0.0, st_Sigma.r);
            st_Out->st_Plot[nTarget].az = st_Out->st_True[nTarget].az + GAUSS(0.0, st_Sigma.az);
            st_Out->st_Plot[nTarget].el = st_Out->st_True[nTarget].el + GAUSS(0.0, st_Sigma.el);
            st_Out->st_PlotLla[nTarget] = f_Pcs_SphToLla(st_Out->st_Plot[nTarget], st_Radar);
        }
        // 클러스터: FOV 안 균등분포, 매 스캔 새로 뽑음
        st_Out->nClutterNum = nClutterNum;
        for (nClutter = 0; nClutter < nClutterNum; nClutter++)
        {
            st_Clutter = &st_Out->st_Clutter[nClutter];
            st_Clutter->r = PCS_CLT_RNG_MIN + ((PCS_CLT_RNG_MAX - PCS_CLT_RNG_MIN) * UNIRAN());
            st_Clutter->az = f_Deg_To_Rad(PCS_CLT_AZI_MIN + ((PCS_CLT_AZI_MAX - PCS_CLT_AZI_MIN) * UNIRAN()));
            st_Clutter->el = f_Deg_To_Rad(PCS_CLT_ELE_MIN + ((PCS_CLT_ELE_MAX - PCS_CLT_ELE_MIN) * UNIRAN()));
            st_Out->st_ClutterLla[nClutter] = f_Pcs_SphToLla(st_Clutter, st_Radar);
        }
    }
}

```

- 1 **p 계산, seed 맞춤**
f_Pcs_CalcSigma 로 p (4.3).
seed 를 12345 로 맞춰 늘 같은 결과
 - 2 **스캔 = 과제 3 표본**
0.1 s 마다 하나, 600개.
레이다 = 그 시각의 플랫폼 위치 · 자세
 - 3 **참값 x**
f_Pcs_EcefToSph: 표적 ECEF 위치 → 레이다
기준 거리 · 방위각 · 고각 (4.5)
 - 4 **플롯 $z = x + v$**
성분마다 GAUSS(0, p) 를 더함.
지도용 위경도도 저장 (4.5)
 - 5 **클러스터**
FOV 최솟값 + 범위 × UNIRAN.
스캔마다 N개 (기본 5) 를 새로 뽑음
- 난수 순서 스캔마다 표적 1 ~ N 의 거리 · 방위각 · 고각,
그다음 클러스터

13

04. 구현과 핵심 코드 | 4.4 f_Pcs_Measure 가 스캔마다 참값 · 플롯 · 클러스터를 만듭니다

발표 대본

측정 모의의 본체인 f_Pcs_Measure 입니다.

왼쪽 번호를 따라가겠습니다.

1번, 처음에 p 를 계산하고 seed 를 12345 로 맞춥니다.

그래서 같은 입력이면 언제나 같은 결과가 나옵니다.

2번, 스캔마다 과제 3 의 표본 하나를 씁니다.

0.1초마다 하나씩 600 스캔이고, 레이다 위치와 자세는 그 시각의 플랫폼입니다.

3번, 표적마다 참값을 레이다 기준 거리, 방위각, 고각으로 바꿉니다.

4번, 성분마다 GAUSS 로 뽑은 잡음을 더하면 플롯이고, 지도에 그리려고 위경도로도 바꿔 둡니다.

5번, 클러스터는 FOV 의 최솟값에 범위와 균등난수를 곱해 더하고, 스캔마다 새로 뽑습니다.

난수는 스캔마다 표적 1번부터 거리, 방위각, 고각 순서로 쓰고, 그다음 클러스터에 씁니다.

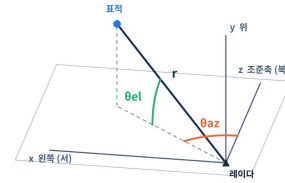
04. 구현과 핵심 코드

4.5 좌표 변환은 참고 소스의 변환 함수를 차례로 부릅니다

```
// ECEF -> NED -> 동체 -> 안테나 -> 좌표표. ③ 참값 x (4.4의 3번)
static STRUCT_Coord_Sph f_Pcs_EcefToSph(const STRUCT_Coord_Rect *st_Ecef,
const ST_TargetState *st_Radar)
{
    const STRUCT_Coord_Attitude *st_Att = &st_Radar->st_Att;
    ...
    st_Ned = f_Trans_Ecef_To_Ned(st_Ecef->x, st_Ecef->y, st_Ecef->z,
    st_Radar->st_PosEcef.x, st_Radar->st_PosEcef.y, st_Radar->st_PosEcef.z,
    st_Radar->st_Lla.Lat, st_Radar->st_Lla.Lon);
    st_Body = f_Trans_Ned_To_Body(st_Ned.x, st_Ned.y, st_Ned.z,
    st_Att->Roll, st_Att->Yaw, st_Att->Pitch);
    st_Ant = f_Trans_Body_To_Ant(st_Body.x, st_Body.y, st_Body.z);
    return f_Trans_Ant_XYZ_To_Sph(st_Ant.x, st_Ant.y, st_Ant.z);
}
```

```
// 좌표표 -> 안테나 -> 동체 -> NED -> ECEF -> LLA. ④ ⑤ 지도용 위경도 (4.4의 4, 5번)
static STRUCT_Coord_Lla f_Pcs_SphToLla(const STRUCT_Coord_Sph *st_Sph,
const ST_TargetState *st_Radar)
{
    ...
    st_Ant = f_Trans_Ant_Sph_To_XYZ(st_Sph->r, st_Sph->az, st_Sph->el);
    st_Body = f_Trans_Ant_To_Body(st_Ant.x, st_Ant.y, st_Ant.z);
    st_Ned = f_Trans_Body_To_Ned(st_Body.x, st_Body.y, st_Body.z, ...);
    st_Ecef = f_Trans_Ned_To_Ecef(st_Ned.x, st_Ned.y, st_Ned.z, ...);
    return f_Trans_Ecef_To_Lla(st_Ecef.x, st_Ecef.y, st_Ecef.z);
}
```

f_Trans_ 함수 = 참고 소스 CoordinateTransform.c 그대로. 레이더 = 그 시각의 플랫폼 (위치 · 자세)



방위각: 조준축에서 왼쪽이 +
지금은 조준축 = 북쪽 → 서쪽이 +

예 표적 1 (대함), t = 0 s
거리 14,148 m · 방위각 -11.54° (동쪽) · 고각 -0.06°

확인 바꿨다가 되돌린 위치 차이: 최대 0.03 mm

14

04. 구현과 핵심 코드 | 4.5 좌표 변환은 참고 소스의 변환 함수를 차례로 부릅니다

발표 대본

3번과 4번에서 쓴 좌표 변환입니다.

위 함수는 참값을 구하고, 아래 함수는 지도에 그리려고 거꾸로 바꿉니다.

둘 다 참고 소스의 변환 함수를 차례로 부르지만 했습니다.

과제 3 의 표적 위치는 지구 중심 기준인 ECEF 입니다.

이것을 레이더 위치 기준 북, 동, 아래 좌표로 바꾸고, 레이더 자세로 동체 좌표, 다시 안테나 좌표로 바꾼 뒤 거리와 두 각도를 구합니다.

오른쪽 그림처럼 방위각은 조준축에서 왼쪽이 플러스입니다.

지금은 조준축이 북쪽이라 서쪽이 플러스입니다.

예를 들어 표적 1, 대함 표적은 처음에 거리 14,148 m, 방위각 마이너스 11.54도로 동쪽에 있고, 고각은 마이너스 0.06도입니다.

고각이 마이너스인 것은 지구가 둥글어서 14 km 떨어진 해면이 레이더 수평면보다 낮기 때문입니다.

참값으로 바꿨다가 아래 함수로 되돌려 보니 위치 차이는 최대 0.03 mm 었습니다.

04. 구현과 핵심 코드

4.6 결과를 모아 오차 표준편차를 구하고 요약과 지도에 씁니다

```

VOID CPlotResult::f_Run(const CSimResult *st_Source, FLOAT64 snr, INT32 nClutterNum)
{
    ...
    1 const INT32 nScanNum = st_Source->f_GetSampleNum() - 1;
    ...
    2 st_Sigma = f_Pcs_CalcSigma(snr);
    st_ScanBuf.resize(static_cast<UINT64>(nScanNum));
    f_Pcs_Measure(st_ScanBuf.data(), st_Source->f_GetSample(0),
        nScanNum, snr, nClutterNum);
    ...
    for (nScan = 0; nScan < nScanNum; nScan++)
    {
        ...
        // 플롯 오차 z - x 는 성분마다 넣은 값을 v 와 같음
        for (nTarget = 0; nTarget < nTargetNum; nTarget++)
        {
            ...
            for (nPart = 0; nPart < 3; nPart++)
            {
                3 sum[nPart] = sum[nPart] + diff[nPart];
                sumSq[nPart] = sumSq[nPart] + (diff[nPart] * diff[nPart]);
            }
            4 st_PlotPointBuf.push_back(f_Meas_ToPoint(&st_Scan->st_PlotLla[nTarget],
                st_Origin));
        }
        ...
    }
    3 st_ErrorStd.r = f_Meas_Std(sum[0], sumSq[0], nPlotNum);
    st_ErrorStd.az = f_Meas_Std(sum[1], sumSq[1], nPlotNum);
    st_ErrorStd.el = f_Meas_Std(sum[2], sumSq[2], nPlotNum);
}

```

1 스캔 600개

표본 601개 - 1 (t=0 ~ 59.9 s)

2 측정 모의는 한 번에

f_Pcs_Measure 가 600 스캔을 모두 채움 (4.4)

3 오차 표준편차

플롯 - 참값 (z - x) 의 합 · 제곱합을 모아 f_Meas_Std (N 으로 나눔)

4 지도 좌표

플롯 · 클러터 위경도 → 플랫폼 처음 위치 기준 동-북

TargetSimUIDlg.cpp f_RunMeasure

```

VOID CTargetSimUIDlg::f_RunMeasure(VOID)
{
    CString st_Text;
    st_PlotResult.f_Run(&st_Result, snr, nClutterNum);
    st_Text.Format(_T("표준편차 p (이론) ..."), ...);
    SetDlgItemText(IDC_PCS_SUMMARY, st_Text);
    st_Plot.f_SetMeasure(&st_PlotResult,
        isShowPlot, isShowClutter);
}

```

요약 2줄 = 이론 p 와 오차 표준편차. f_SetMeasure 로 지도를 다시 그림 (클러터 → 플롯 → 궤적 순서)

15

04. 구현과 핵심 코드 | 4.6 결과를 모아 오차 표준편차를 구하고 요약과 지도에 씁니다

발표 대본

구현의 마지막은 화면 쪽에서 결과를 정리하는 CPlotResult 의 f_Run 입니다.

1번, 스캔 수는 표본 수 빼기 1, 즉 600 입니다.

2번, f_Pcs_Measure 를 한 번 불러 600 스캔을 모두 채웁니다.

3번, 플롯에서 참값을 뺀 오차를 성분마다 합과 제곱합으로 모아 마지막에 표준편차를 구합니다.

4번, 플롯과 클러터의 위경도는 지도 좌표로 바꿔 둡니다.

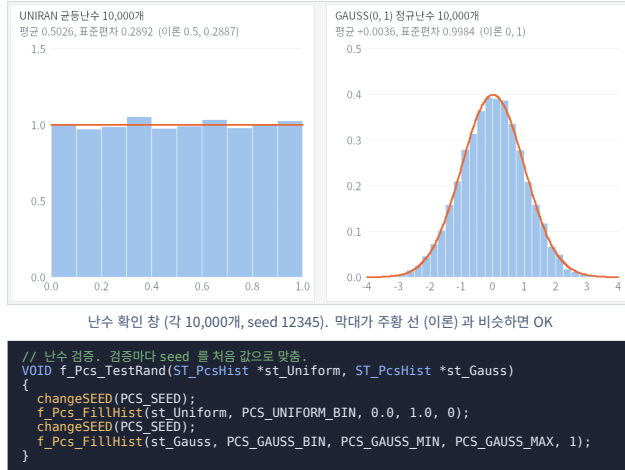
오른쪽 아래 f_RunMeasure 가 이 함수를 부르고, 이론 p 와 오차 표준편차를 요약 두 줄로 쓴 뒤 지도를 다시 그림니다.

지도는 클러터, 플롯, 궤적 순서로 그려서 점이 선을 가리지 않습니다.

여기까지가 구현이고, 이제 결과를 보겠습니다.

05. 실습 결과

5.1 실습 1 난수 UNIRAN 과 GAUSS 가 이론 분포와 맞았습니다



UNIRAN 균등난수

막대 높이	0.97 ~ 1.05	기준 1
평균	0.503	기준 0.5
0 과 1	안 나옴	(0, 1) 구간

GAUSS 정규난수

평균	+0.004	기준 0
표준편차	0.998	기준 1

막대 높이 = 개수 ÷ (10,000 × 막대 폭)
이론 선: UNIRAN 은 높이 1, GAUSS 는 N(0, 1) 곡선

UNIRAN PASS

GAUSS PASS

16

05. 실습 결과 | 5.1 실습 1 난수 UNIRAN 과 GAUSS 가 이론 분포와 맞았습니다

발표 대본

다섯 번째, 실습 결과입니다.

실습 1 은 난수입니다.

잡음과 클러터를 모두 난수로 만들기 때문에 난수부터 확인했습니다.

왼쪽은 화면의 난수 확인 창입니다.

아래 f_Pcs_TestRand 가 seed 를 12345 로 맞추고, UNIRAN 과 GAUSS 를 각각 10,000개 뽑아 막대로 썹니다.

오른쪽 숫자를 보시면, UNIRAN 은 막대 높이가 0.97 에서 1.05 사이로 1 근처에 고르게 나왔고, 평균은 0.503 입니다.

GAUSS 는 평균 0.004, 표준편차 0.998 로 이론 곡선과 겹칩니다.

두 함수 모두 통과입니다.

05. 실습 결과

5.2 실습 2 플롯 오차의 표준편차가 이론값 p 와 맞았습니다

첫 플롯 표적 1, $t=0$ s, 거리참값 x
14,147.51 m

+

잡음 $v = \text{GAUSS}(0, 4.74 \text{ m})$
-2.12 m
(UNIRAN 12개 합 5.5538 - 6) $\times$ 4.74 m

=

플롯 $z = x + v$
14,145.40 m

거리

이론 4.74 m

4.80 m

 $\times 1.013$

방위각

이론 0.478°

0.476°

 $\times 0.996$

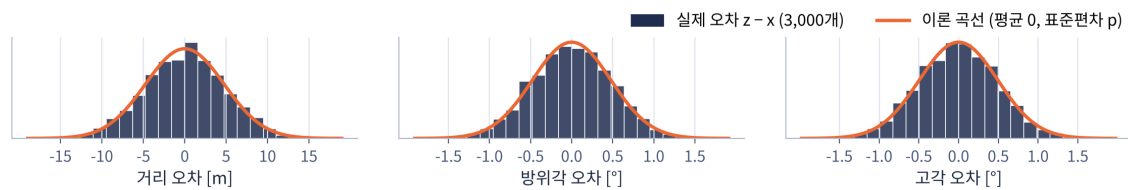
고각

이론 0.494°

0.493°

 $\times 0.998$ 실제 표준편차 (3,000개) $\div$ 이론 모두 1 근처, 기준 0.9 ~ 1.1. 3,000개면 $\pm 1.3\%$ 흔들림은 정상

PASS



17

05. 실습 결과 | 5.2 실습 2 플롯 오차의 표준편차가 이론값 p 와 맞았습니다

발표 대본

실습 2, 플롯입니다.

위는 첫 플롯입니다.

표적 1 의 첫 거리 참값은 14,147.51 m 입니다.

처음 뽑은 GAUSS 에서 UNIRAN 12개의 합이 5.5538 이고, 6 을 빼고 4.74 m 를 곱하면 잡음은 마이너스 2.12 m, 더하면 플롯은 14,145.40 m 입니다.

이렇게 표적 5개로 600 스캔 동안 플롯 3,000개를 만들고, 플롯에서 참값을 빼서 오차의 표준편차를 구했습니다.

화면 요약 둘째 줄과 같은 값입니다.

거리는 4.80 m 로 이론값의 1.013배, 방위각은 0.996배, 고각은 0.998배입니다.

3,000개면 1.3 퍼센트 정도 흔들리는 것이 정상이라 잘 맞습니다.

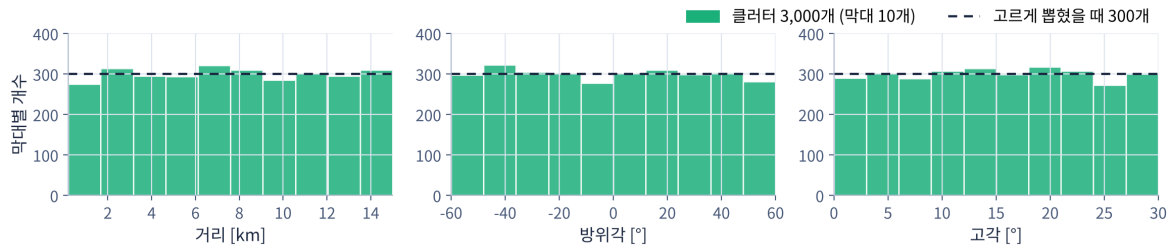
아래 히스토그램도 이론 곡선과 겹칩니다.

05. 실습 결과

5.3 실습 3 클러스터 3,000개가 FOV 안에 고르게 퍼졌습니다

```
st_Clutter->r = PCS_CLT_RNG_MIN + ((PCS_CLT_RNG_MAX - PCS_CLT_RNG_MIN) * UNIRAN());
```

방위각 · 고각도 같은 식 (4.4) 스캔마다 5개



거리 카이제곱

5.91

< 21.67

방위각 카이제곱

5.09

< 21.67

고각 카이제곱

5.53

< 21.67

FOV 밖 0개

스캔마다 5개, 방위각 + 는 모두 서쪽

카이제곱 = $\sum (\text{개수} - 300)^2 \div 300$. 21.67 = 막대 10개일 때 우연히 넘을 확률 1% 인 값

18

05. 실습 결과 | 5.3 실습 3 클러스터 3,000개가 FOV 안에 고르게 퍼졌습니다

발표 대본

실습 3, 클러스터입니다.

위 코드처럼 범위의 최솟값에, 범위 폭과 균등난수를 곱한 값을 더합니다.

방위각과 고각도 같은 식입니다.

스캔마다 5개, 모두 3,000개를 만들었고 FOV 밖은 하나도 없습니다.

그래프는 성분마다 막대 10개로 나눈 개수입니다.

고르게 뿔었다면 막대마다 300개쯤이고, 점선이 그 300 입니다.

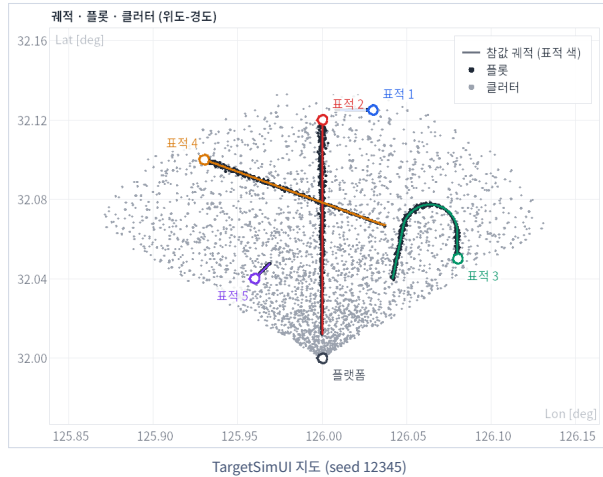
고른 정도는 카이제곱으로 봤습니다.

거리 5.91, 방위각 5.09, 고각 5.53 으로 모두 기준 21.67 보다 작아서 고르다고 볼 수 있습니다.

방위각이 플러스인 클러스터는 모두 서쪽에 찍혀 부호도 맞았습니다.

05. 실습 결과

5.4 참값 · 플롯 · 클러스터를 LLA 지도에 색으로 나눠 그렸습니다



선 = 참값 (표적마다 색)

표적이 실제로 지나간 길. 이름표가 있는 곳이 시작점

검은 점 = 플롯 3,000개

선 바로 옆에 붙어 있으면 정상. 표적 3의 선회도 보인다

회색 점 = 클러스터 3,000개

부채꼴 범위 (FOV) 안에만 있으면 정상

지도용 LLA 변환

구좌표 → 안테나 → 등체 → NED → ECEF → LLA (4.5의 f_Pcs_SphToLla)

레이다 가까이 클러스터가 촘촘한 이유

거리를 고르게 뽑아서, 넓이가 작은 가까운 곳에 몰려 보인다 (정상)

19

05. 실습 결과 | 5.4 참값 · 플롯 · 클러스터를 LLA 지도에 색으로 나눠 그렸습니다

발표 대본

마지막으로 결과를 LLA 지도에 그렸습니다.

과제 3의 궤적 지도에 겹쳐 그렸고, 4.5의 f_Pcs_SphToLla로 위도, 경도, 고도로 바꿉니다.

선이 참값, 즉 표적 5개가 실제로 지나간 길입니다.

과제 3처럼 표적마다 색이 다르고, 이름표가 있는 곳이 시작 위치입니다.

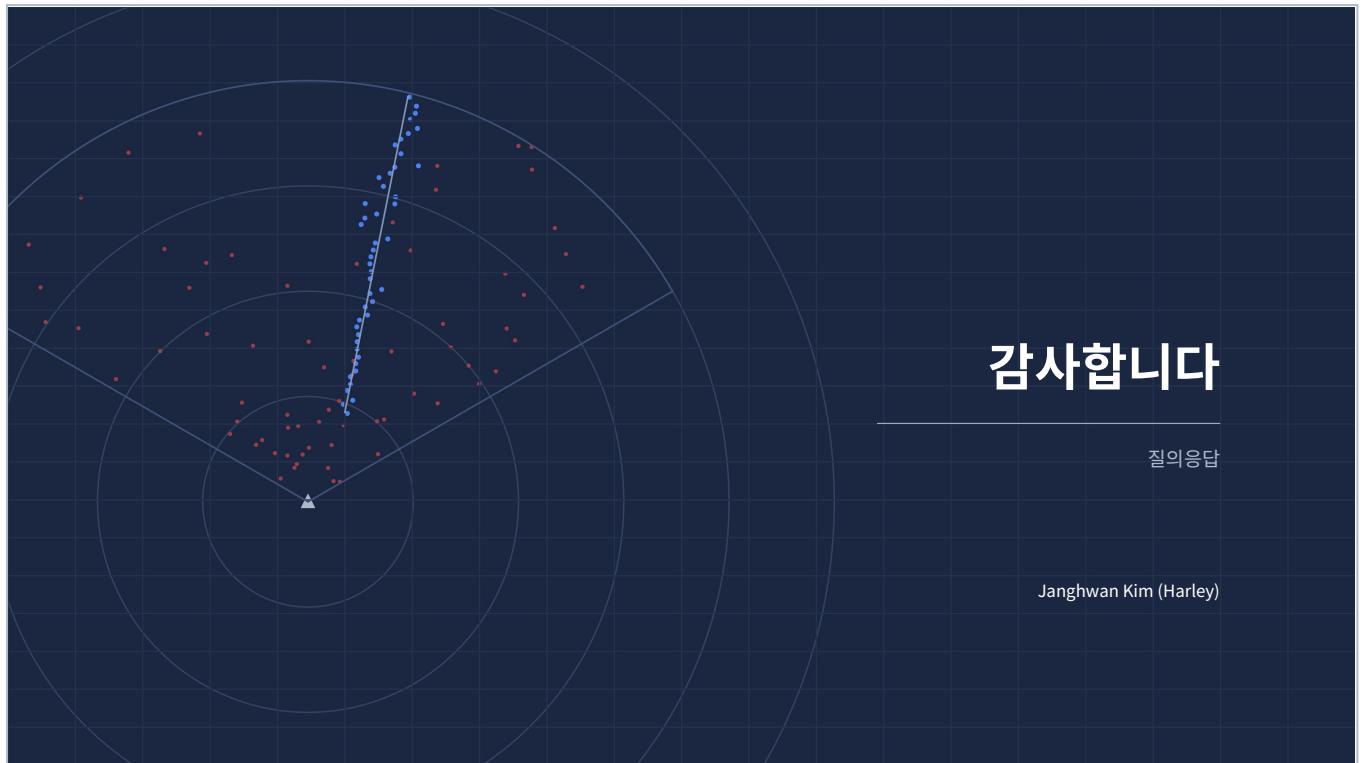
검은 점은 플롯 3,000개로, 선 바로 옆에 붙어 있습니다.

3번 표적이 크게 도는 모습도 보입니다.

회색 점은 클러스터 3,000개이고, 부채꼴 범위 안에만 있습니다.

레이다 가까이 클러스터가 촘촘한 것은 오류가 아닙니다.

거리를 고르게 뽑아서, 넓이가 작은 가까운 곳에 점이 몰려 보이는 것입니다.



마무리

발표 대본

들어주셔서 감사합니다.

질문 있으시면 말씀해 주세요.